To delete a `Musician`, you can either go to the Change screen and push the Delete button, or on the listing screen, you can select one or more `Musician` objects using the check boxes and then choose Delete Selected Musicians from the Action drop-down menu.

All these pages get created for you because you wrote a simple `admin.ModelAdmin` class. That's a lot of benefit for very little work, and what you've seen so far is merely the default. You can also customize the class to control the appearance and features of the page.

5.2 Customizing a listing page

The Django Admin's class listing page shows each of the objects in the database for the corresponding class. By default, each object gets represented using the class's `.__str__()` method. You can change this behavior by adding the `list_display` attribute to the `admin.ModelAdmin` class, which controls which columns are shown on the listing page. The attribute takes an iterable (usually a `tuple`) with an item for each desired column. Each item is a string: naming an object attribute, an argument-less object callable, or a callable on the `admin.ModelAdmin` class itself. In the case of a callable, the result returned from the method is the value for the column.

Let's replace the default display for the `MusicianAdmin` class. Add the `list_display` attribute to the `MusicianAdmin` class, setting it to a `tuple` containing `("id", "last_name", "show_weekday")`. The first two items in this `tuple` result in a `Musician` object's ID and `last_name` fields being used as columns. The `"show_weekday"` value, in this case, is for a callable, which will be added to the class.

My `.show_weekday()` callable is a bit silly—it shows the day of the week the artist was born and can be implemented on either the `Musician` class or the `admin.ModelAdmin` class. On the `Musician` class, it needs to be a method without any arguments. As I only want this "feature" in the Django Admin, it is a best practice to put it on the `MusicianAdmin` class instead. In this case, the method takes a single argument, the object to be displayed, and gets called once for each object in the listing. The return value of the method populates the row in the column.

The title of the column for a callable is based on the name of the callable. Underscores are turned into spaces, and words are displayed in small caps. You can separate the name of your callable from the title name with one unusual thing. You may have heard that everything in Python is an object, even functions—well, this goes for methods on objects as well. Like all objects, you can assign attributes to a method. To me, this feels sort of strange, and there aren't many use cases for it in the real world, but Django uses this language feature. The `.show_weekday()` method gets the optional string attribute `.short_description`, which, if present, gets used as the column title instead of the method name.

Listing 5.2 shows the new version of `MusicianAdmin` that defines the `list_display` attribute and the `.show_weekday()` method, including the `.short_description` attribute that renames the column title. Update your admin.py with this content.